

МИНОБРНАУКИ РОССИИ

Федеральное государственное автономное образовательное
учреждение высшего образования
«Южный федеральный университет»

Институт математики, механики и компьютерных
наук им. И. И. Воровича

Кафедра информатики и вычислительного эксперимента

Пушкова Анастасия Дмитриевна

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ НА
ОСНОВЕ НЕЙРОННЫХ СЕТЕЙ ДЛЯ ОЦИФРОВКИ
НОТНЫХ ПАРТИТУР ПО ИЗОБРАЖЕНИЯМ И ИХ
ПОСЛЕДУЮЩЕЙ ОБРАБОТКИ**

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по направлению подготовки

02.03.02 — Фундаментальная информатика и информационные технологии

Научный руководитель —

доцент, к. ф.-м. н. Андреева Евгения Михайловна

Допущено к защите:

руководитель направления 02.03.02 _____ Михалкович С. С.

Ростов-на-Дону – 2026

Задание на выпускную квалификационную работу бакалавра

Направление подготовки: 02.03.02 – Фундаментальная информатика и информационные технологии.

Студент: Пушкова А.Д.

Научный руководитель: доцент Андреева Е.М.

Год защиты: 2026

Тема работы: РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ НА ОСНОВЕ НЕЙРОННЫХ СЕТЕЙ ДЛЯ ОЦИФРОВКИ НОТНЫХ ПАРТИТУР ПО ИЗОБРАЖЕНИЯМ И ИХ ПОСЛЕДУЮЩЕЙ ОБРАБОТКИ

Цель работы: разработка интеллектуальной системы на основе нейронных сетей для оцифровки нотных партитур по изображениям и их последующей обработки

Задачи:

1. Анализ существующих методов для распознавания нотных партитур по изображениям.
2. Разработка оптимальной архитектуры системы, сочетающей детерминированные алгоритмы с нейронными сетями.
3. Реализация детерминированного алгоритма для определения позиций нотных линий на изображении.
4. Поиск подходящего набора данных и его предобработка для работы с выбранной архитектурой нейронной сети.
5. Обучение модели нейронной сети для распознавания музыкальных символов на изображении.
6. Разработка алгоритма для объединения данных, извлеченных из изображения вышеописанными методами, и формирования списка нот.
7. Разработка подсистемы, преобразующей сформированный список нот в аппликатуру для классической шестиструнной гитары.

Научный руководитель

Андреева Е.М.

Студент

Пушкова А.Д.

Отзыв научного руководителя

на выпускную квалификационную работу Пушковой А.Д.

РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ НА ОСНОВЕ НЕЙРОННЫХ СЕТЕЙ ДЛЯ ОЦИФРОВКИ НОТНЫХ ПАРТИТУР ПО ИЗОБРАЖЕНИЯМ И ИХ ПОСЛЕДУЮЩЕЙ ОБРАБОТКИ

по направлению 02.03.02 – Фундаментальная информатика
и информационные технологии

Выпускная квалификационная работа Пушковой А.Д. посвящена разработке интеллектуальной системы на основе нейронных сетей, способной распознавать нотные партитуры для гитары по изображениям (скан-копия, фотография) и формировать для каждой ноты указания по выбору оптимальных струны и позиции на грифе, то есть аппликатуру для прочитанных партитур.

В рамках данной работы Пушковой А.Д. были изучены существующие методы распознавания нотных партитур по изображениям. Разработана оптимальная архитектура системы, сочетающей детерминированные алгоритмы с нейронными сетями. Реализован детерминированный алгоритм для определения позиций нотных линий на изображении. Проведён поиск подходящего набора данных, выполнена его предобработка для работы с выбранной архитектурой нейронной сети, и обучена модель нейронной сети для распознавания музыкальных символов на изображении. Разработан алгоритм для объединения данных, извлечённых из изображения вышеописанными методами, и формирования списка нот. Также разработана подсистема, преобразующая сформированный список нот в аппликатуру для классической шестиструнной гитары.

Результатом работы стала интеллектуальная система, объединяющая в себе детерминированные алгоритмы определения нотных линий, нейросетевые модели распознавания музыкальных символов и алгоритмы формирования аппликатуры. Система способна обрабатывать скан-копии и фотографии нотных листов формата А4 с плоского оригинала при допустимом наклоне, обеспечивая корректное распознавание печатной нотации.

Степень оригинальности выпускной квалификационной работы 95,67% соответствует требованиям, предъявляемым к выпускным квалификационным работам данного уровня образования.

В ходе выполнения работы Пушкова А.Д. показала способность самостоятельно осваивать новый материал, проводить сравнительный анализ подходов к разработке и успешно применять полученные знания на практике при создании полноценной интеллектуальной системы. Выпускная

квалификационная работа на тему «Разработка интеллектуальной системы на основе нейронных сетей для оцифровки нотных партитур по изображениям и их последующей обработки» заслуживает оценки «отлично» (95 баллов) и может быть допущена к защите.

Кандидат физико-математических наук,
доцент кафедры ИВЭ

Андреева Е.М.

СПРАВКА

о результатах проверки текстового документа
на наличие заимствований

Федеральное государственное автономное
образовательное учреждение высшего
образования «Южный федеральный
университет»

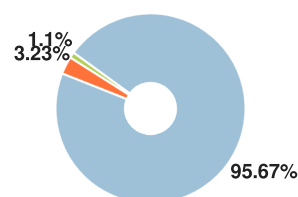
ПРОВЕРКА ВЫПОЛНЕНА В СИСТЕМЕ АНТИПЛАГИАТ.ВУЗ

Автор работы: Пушкова Анастасия Дмитриевна
**Самоцитирование
рассчитано для:** Пушкова Анастасия Дмитриевна
Название работы: РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ НА ОСНОВЕ НЕЙРОННЫХ СЕТЕЙ ДЛЯ
ОЦИФРОВКИ НОТНЫХ ПАРТИТУР ПО ИЗОБРАЖЕНИЯМ И ИХ ПОСЛЕДУЮЩЕЙ ОБРАБОТКИ
Тип работы: Выпускная квалификационная работа
Подразделение: Институт математики, механики и компьютерных наук им. И.И. Воровича

РЕЗУЛЬТАТЫ

СОВПАДЕНИЯ	3.23%
ОРИГИНАЛЬНОСТЬ	95.67%
ЦИТИРОВАНИЯ	1.1%
САМОЦИТИРОВАНИЯ	0%
ИИ-КОНТЕНТ	0%

ДАТА ПОСЛЕДНЕЙ ПРОВЕРКИ: 09.06.2026



Структура документа: Проверенные разделы: основная часть с.13-33, титульный лист с.1, содержание с.2, введение с.3-13, выводы с.34
Модули поиска: Профессиональная лексика. Медицина; Профессиональная лексика. АПК и биотех; Профессиональная лексика. Юриспруденция; Переводные заимствования; ИПС Адилет; Цитирование; Шаблонные фразы; СМИ России и СНГ; IEEE; Патенты СССР, РФ, СНГ; Перефразирования по базе публикаций открытого доступа PubMed; PubMed; Переводные заимствования IEEE; Публикации eLIBRARY; Публикации РГБ (переводы и перефразирования); Сводная коллекция научных работ Беларуси; Коллекция НБУ; Перефразирования по Коллекции открытых публикаций международных издательств; Публикации РГБ; СПС ГАРАНТ: аналитика; Публикации eLIBRARY (переводы и перефразирования); Перефразирования по коллекции IEEE; Переводные заимствования по Коллекции открытых публикаций международных издательств; Кольцо вузов; Перефр...

Работу проверил: Андреева Евгения Михайловна

ФИО проверяющего

Дата подписи:

Подпись проверяющего



Чтобы убедиться
в подлинности справки, используйте QR-код,
который содержит ссылку на отчет.

Ответ на вопрос, является ли обнаруженное заимствование
корректным, система оставляет на усмотрение проверяющего.
Предоставленная информация не подлежит использованию
в коммерческих целях.

Оглавление

Введение.....	3
Постановка задачи.....	5
1. Оптическое распознавание музыки.....	7
2. Анализ существующих методов для распознавания нотных партитур по изображениям и разработка архитектуры системы	10
3. Реализация алгоритма для определения позиций нотных линий на изображении.....	12
4. Подготовка датасета.....	16
5. Обучение модели нейронной сети.....	20
6. Интеграция результатов детекции нот и линий для формирования списка нот	26
7. Алгоритм построения аппликатуры для шестиструнной гитары методом динамического программирования.....	29
Использование ИИ в работе	33
Заключение.....	34
Список литературы	35

Введение

Музыка – один из самых древних видов искусства, неотъемлемая часть культуры. В современности мы способны воспроизвести музыку, которую люди слушали сотни лет назад благодаря тому, что она была сохранена с помощью музыкальной нотации – системы фиксации музыки с помощью письменных знаков.

В XVII веке система записи музыки принимает вид, которая используется и по сей день. В какой-то степени эта система напоминает запись языка: существует список символов, каждый из которых имеет какой-то смысл, но в отличие от типичных алфавитных языков, значение имеет не только последовательность символов, но и их положение в пространстве друг относительно друга.

В этой работе исследуется распознавание нотных партитур (графических записей музыкального произведения) по изображениям или оптическое распознавание музыки – область машинного обучения, целью которой является «научить» компьютер читать нотные листы так, как это делает человек.

Эта сфера начала развиваться в 1960-х с работами сотрудников и студентов Массачусетского технологического института, но настоящий прогресс получила только с распространением глубокого обучения и сверточных нейронных сетей, способных куда эффективнее распознавать музыкальные символы. Тем не менее, и в настоящее время количество программного обеспечения, решающего эту задачу, крайне ограничено и в большей степени сосредоточено на более распространенных музыкальных инструментах, преимущественно фортепиано.

Эта работа ставит своей целью создание системы, способной распознавать нотные листы по изображениям, в частности листы гитарной музыки (для классической шестиструнной гитары), так как она имеет ряд особенностей, присущих именно этому инструменту, а также обрабатывать

полученные данные для формирования аппликатуры (расположения нот по ладам и струнам).

Постановка задачи

Целью данной работы является разработка интеллектуальной системы на основе нейронных сетей, способной распознавать нотные партитуры для гитары по изображениям (скан-копия, фотография) и формировать для каждой ноты указания по выбору оптимальных струны и позиции на грифе, то есть аппликатуру для прочитанных партитур. Для выполнения поставленной цели были сформулированы следующие задачи:

1. Анализ существующих методов для распознавания нотных партитур по изображениям.
2. Разработка оптимальной архитектуры системы, сочетающей детерминированные алгоритмы с нейронными сетями.
3. Реализация детерминированного алгоритма для определения позиций нотных линий на изображении.
4. Поиск подходящего набора данных и его предобработка для работы с выбранной архитектурой нейронной сети.
5. Обучение модели нейронной сети для распознавания музыкальных символов на изображении.
6. Разработка алгоритма для объединения данных, извлеченных из изображения вышеописанными методами, и формирования списка нот.
7. Разработка подсистемы, преобразующей сформированный список нот в аппликатуру для классической шестиструнной гитары.

В рамках данной работы приняты следующие допущения:

- данные, с которыми работает система, представляют собой скан-копию или фотографию нотного листа А4 с плоского оригинала (без искривлений страницы, наклон допустим);
- данные представляют собой печатные листы (не работает с рукописными);

- линии нотного стана на изображении ориентированы приблизительно горизонтально (может быть наклон с листом), без поворота на 90° .

1. Оптическое распознавание музыки

Распознавание нотных партитур по изображениям, или оптическое распознавание музыки (Optical Music Recognition, OMR), – это сложная и активно развивающаяся междисциплинарная область, совмещающая различные методики машинного обучения и музыкальную теорию.

OMR часто сравнивается с OCR (Optical Character Recognition, оптическое распознавание символов), но несмотря на кажущиеся сходства, эти сферы требуют разительно разного подхода к анализу изображений. Выделим ключевые особенности работы с музыкальной нотацией и отличия от OCR:

- необходимость работы в двухмерном пространстве – для нот и других музыкальных знаков имеет критическое значение не только их порядок слева направо, но также и их расположение по вертикальной оси;
- потребность сохранения контекста предыдущих символов куда ниже, чем при работе с текстом, так как большая часть из них независима;
- музыкальные символы очень разнообразны, значительно отличаясь по размерам и форме: от небольших точек до линий в большую часть ширины листа [1].

Нотные партитуры имеют достаточно специфическую структуру, как приведено на рисунке 1. На нем синим выделены нотные линии, в зеленых прямоугольниках индивидуальные ноты, а в красных – другие нотные символы.

В рассматриваемой области перед программой ставится задача локализовать и определить тип каждого музыкального символа на изображении, понять, что их связывает между собой и связывает ли вообще. Дополнительную сложность добавляет количество различных шрифтов и стилей написания музыки, в которых одни и те же символы могут выглядеть очень по-разному (рис. 2).

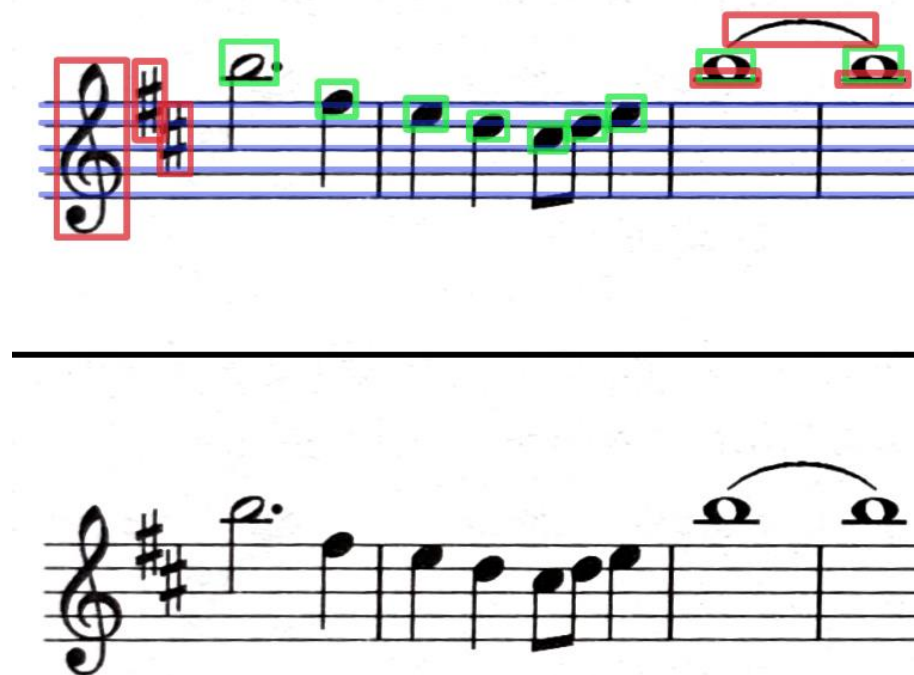


Рисунок 1. Изображение нотной партитуры с отмеченными знаками музыкальной нотации

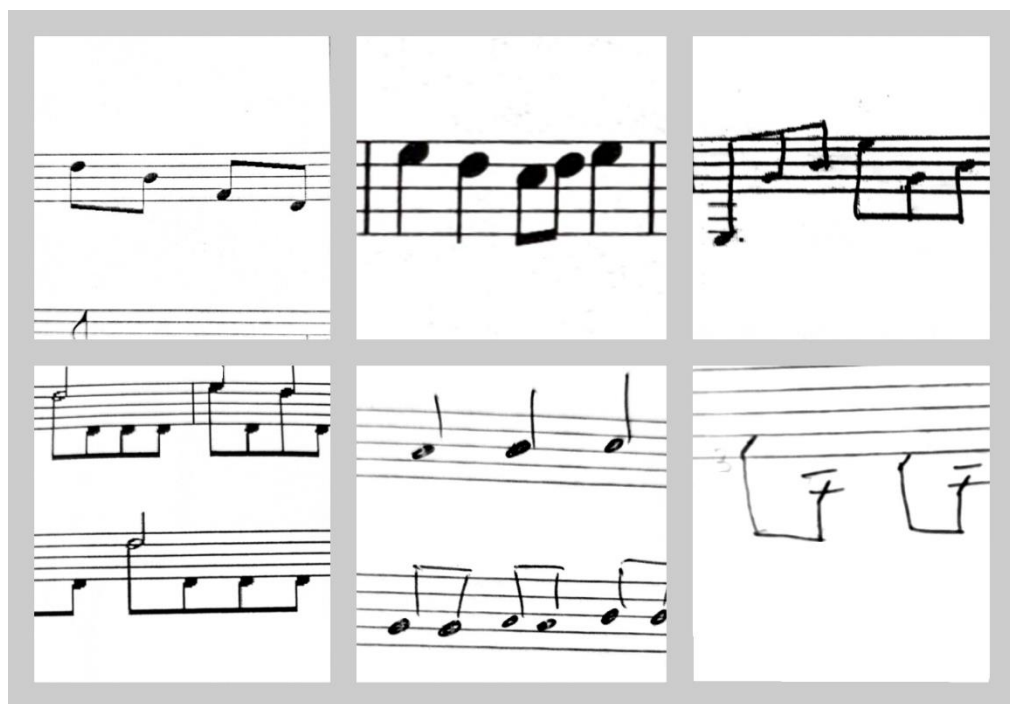


Рисунок 2. Примеры разных вариантов написания нот

Вероятно, самым важным элементом звукозаписи являются ноты. Для определения высоты конкретной ноты необходимо понимать её положение относительно нотных линий, то есть система должна детектировать как местонахождение всех нот, так и местонахождение линий, а потом

анализировать эти данные и соотносить друг с другом. Длительность ноты зависит в основном от вида её штиля (количества флажков на нем), за исключением полных и половинных нот (рис. 3).



Рисунок 3. Примеры нот разных длительностей (слева направо – полная, половинная, четвертная, восьмая, шестнадцатая)

Все перечисленные факторы обуславливают то, что для задачи оптического распознавания музыки не существует простого решения: необходимо сочетание различных направлений программирования. Сверточные нейросети можно использовать для определения знаков на нотном листе, но они сами по себе не способны различить 2 одинаково выглядящие ноты в разных частях изображения как ноты разной высоты.

В этой работе преимущественно рассматривается нотация гитарной музыки, имеющая несколько особенностей: за редким исключением, используется только скрипичный ключ (нет басового, альтового и т. п.), используется большое количество добавочных линий сверху и снизу от нотного стана.

2. Анализ существующих методов для распознавания нотных партитур по изображениям и разработка архитектуры системы

Существует ограниченное количество программного обеспечения (ПО), имеющего функции оптического распознавания нотных листов, во многом это объясняется достаточно небольшим спросом на подобные программы по сравнению с другими областями компьютерного зрения, такими как OCR, распознавание лиц и т. д.

Большая часть искомого ПО является коммерческим, например, Dorico, PhotoScore, Soundslice, Capella Software. Все эти программы предоставляют функции сканирования PDF-файлов и преобразования их в MusicXML и/или специфический формат используемого приложения.

Несмотря на то, что некоторые из перечисленных программных продуктов заявляют о достижении точности до 100%, отсутствие прозрачности в методологии получения этих результатов делают невозможным полноценное сравнение эффективности систем [2]. Кроме того, так как перечисленное здесь ПО является коммерческим, оно поставляется в закрытом исходном коде, что делает невозможным использование его в качестве основы для реализации или заимствования программных решений.

Проекты с открытым исходным кодом в основном являются учебными и редко достигают состояния продукта, готового к распространению. Audiveris – один из немногих таких продуктов, получивший полноценный релиз и активно получающий поддержку по сей день.

Audiveris – программа для оптического распознавания музыки с открытым кодом. Она сочетает в себе 2 системы: одну для распознавания нотных листов и другую для редактирования полученных результатов до сохранения в формате MusicXML [3].

Процесс распознавания состоит из 20 шагов, применяющих множество различных техник для разных элементов. На первом этапе обрабатывается весь

лист: происходит бинаризация, масштабирование и строится сетка, которая помогает системе ориентироваться на листе. Далее выделяются все объекты:

1. Выделяются основные элементы: линии станов, тактовые черты, штили и ребра нот.
2. Происходит распознавание текста на листе с помощью внешнего ПО Tesseract.
3. Все оставшиеся элементы становятся претендентами на музыкальные символы.

Большинство музыкальных символов определяются с помощью нейросети, но для других объектов более эффективными оказались классические методы компьютерного зрения, например, поиск по соответствию шаблонам [4].

Разработка архитектуры системы

С учетом проведенного анализа было принято решение сочетать в разрабатываемой системе распознавание с помощью нейронных сетей с классическими методами CV (computer vision, компьютерное зрение), такими как горизонтальная проекция и преобразование Хафа, и другими детерминированными алгоритмами для обработки данных.

Для обнаружения позиций нотных линий использование нейросетей оказалось неоптимальным, так как они бы требовали создания крайне нестабильных (длинных и тонких) ограничивающих прямоугольников (bounding box). Поэтому для решения этой задачи был разработан алгоритм, сочетающий преобразование Хафа с алгоритмом детектирования пиков на нормированном горизонтальном профиле.

В то же время для определения других объектов, таких как знаки нот, знаки альтерации и др. было решено использовать нейросетевую модель, способную их классифицировать и определить их позицию.

3. Реализация алгоритма для определения позиций нотных линий на изображении

Для нахождения нотных линий на изображении партитуры был разработан следующий алгоритм:

1. Бинаризация и инвертирование изображения для упрощения обработки.
2. На бинарном изображении запускается вероятностное преобразование Хафа для поиска линий и определения возможного угла наклона.
3. Выпрямление, построение горизонтальной проекции и определение пиков на ней.

Вероятностное преобразование Хафа

Преобразование Хафа – это алгоритм поиска прямых линий на изображении, где выделяются границы изображенных объектов, и каждая граничная точка «голосует» за все возможные прямые. Возвращает прямые, достигающие определенного порога уверенности.

Вероятностное преобразование Хафа – оптимизированная версия классического преобразования Хафа. В отличие от стандартного метода, она работает не со всеми граничными точками, а со случайной выборкой, что ускоряет вычисления, а также сразу возвращает отрезки линий, а не бесконечные прямые.

Программа находит линии на бинаризованном изображении с помощью функции `HoughLinesP` из библиотеки `OpenCV`, реализующей вероятностное преобразование Хафа с дополнительной оптимизацией (найденные отрезки удаляются из дальнейшего поиска). Далее вычисляется угол наклона каждой линии и отбираются те, у которых его абсолютное значение не превышает 10° . Результирующим углом поворота изображения является медиана вычисленных углов (лист. 1).

Листинг 1. Фрагмент кода с поиском угла наклона

```
lines = cv2.HoughLinesP(  
    image,  
    rho=1, theta=np.pi / 180,  
    threshold=HOUGH_THRESHOLD,  
    minLineLength=MIN_LINE_LENGTH,  
    maxLineGap=MAX_LINE_GAP  
)  
if lines is None:  
    return image  
angles = []  
for x1, y1, x2, y2 in lines[:, 0]:  
    a = np.degrees(np.arctan2(y2 - y1, x2 - x1))  
    if abs(a) < 10:  
        angles.append(a)  
if not angles:  
    return image  
angle = float(np.median(angles))
```

При повороте все пиксели оригинального изображения сохраняются и дополняются «фоновым» цветом для того, чтобы избежать потери данных (рис. 4).



Рисунок 4. Пример работы алгоритма выравнивания

Работа с горизонтальной проекцией

После выравнивания изображения для него строится нормализованная горизонтальная проекция. Нормализация нужна для повышения стабильности

алгоритма и нейтрализации влияния размера конкретного образца. На построенной проекции находятся пики – горизонтальные ряды с наибольшим количеством черных пикселей (белых пикселей на рабочем изображении) (лист. 2).

Листинг 2. Построение горизонтальной проекции и поиск пиков на ней

```
profile = binary_aligned.sum(axis=1) / binary_aligned.shape[1]
detected_peaks, _ = find_peaks(
    profile,
    height=profile.max()*0.4,
    distance=PEAK_MIN_DISTANCE,
    prominence=PEAK_PROMINENCE * profile.max()
)
```

На рисунке 5 изображен пример обнаружения линий нотного стана на изображении разработанным алгоритмом, а также соответствующий график, отображающий плотность пикселей на строку, на котором ищутся пики.

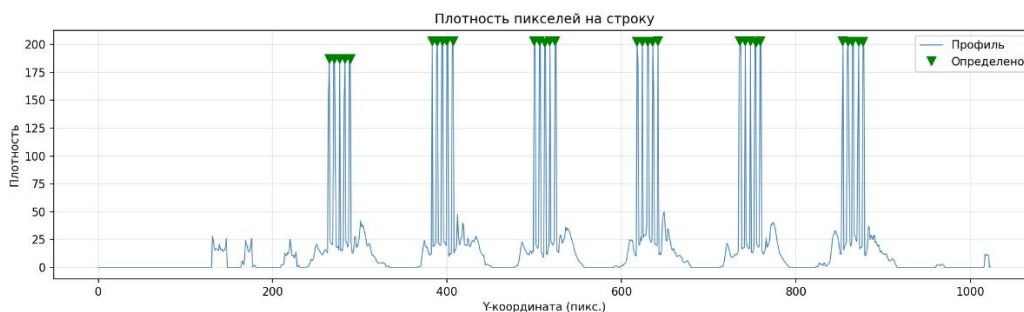


Рисунок 5. Обнаружение линий нотного стана (А. Рамирес «Странники», фрагмент кантаты «Рождество Господне»)

Обнаруженные проблемы алгоритма:

- Иногда происходит ложное срабатывание для разметки, представляющей собой горизонтальные линии (рис. 6). Решение: дополнительная проверка при объединении обнаруженных линий в станы.



Рисунок 6. Пример ложного срабатывания алгоритма

- Чувствителен к искажениям, например, к изгибу страницы на фотографии. Решение требует введения более сложного алгоритма, существующая система работает только с изображениями без искажений.

Разработанный алгоритм был запущен на всех изображениях датасета для тестирования его работы и продемонстрировал точность 97,3%. Детальные значения разных метрик представлены в таблице 1.

Таблица 1. Значения метрик (5218 изображений, 35773 линий)

Precision	Recall	Macro F1	Micro F1
97.0%	99.9%	98.3%	97.3%

4. Подготовка датасета

Был использован датасет **DOReMi**. Этот датасет содержит набор данных для исследований в области оптического распознавания музыки. DOReMi является результатом совместной работы команды Dorico компании Steinberg и сотрудников Лондонского университета королевы Марии (Элоны Шатри и Джорджа Фазекас). Данные представляют собой изображения, сгенерированные с помощью Dorico (программы для нотной записи от Steinberg), и включают в себя большее количество объектов, классов и различных случаев нотации, в том числе варианты, которые обычно не встречаются в реальной музыке.

Часть набора данных, не защищенная авторским правом, содержит 5218 изображений нотных страниц размера 2475×3504 , разметка для которых представлена в 5 разных форматах:

1. **Dorico project** – при сохранении партитур в Dorico используется формат файлов .dorico, сохраняющий данные о проекте, включая макеты, настройки инструментов, исполнителей, события (по аналогии с MIDI) и др.
2. **MusicXML** – широко используемый формат, хранит информацию в файлах .xml, позволяет переносить данные между разными программами для создания нот.
3. **OMR metadata** – файлы формата .xml содержат информацию об ограничивающих рамках, пиксельных масках и ID для каждого элемента, о взаимосвязях между ними по стандарту MUNG, музыкальных атрибутах (длительность такта, высота тона), а также Dorico-события.
4. **Musical Instrument Digital Interface (MIDI)** – формат, хранящий информацию о воспроизведении музыки, а не нотных листах. Содержит такую информацию как: длительность и высота нот, временные параметры и др.

5. **Music Encoding Initiative (MEI)** – выделяет четыре отдельных аспекта нотной записи: логический, жестовый, визуальный и аналитический. Включает информацию о высоте и длительности нот, характеру игры, нотных партитурах, а также анализе и комментариях о всех описанных частях.

Данные содержат 94 класса объектов с выраженным разбросом частоты встречаемости: от редких знаков альтерации (1 пример) до 247741 примеров заполненных нот [5] (рис. 7).

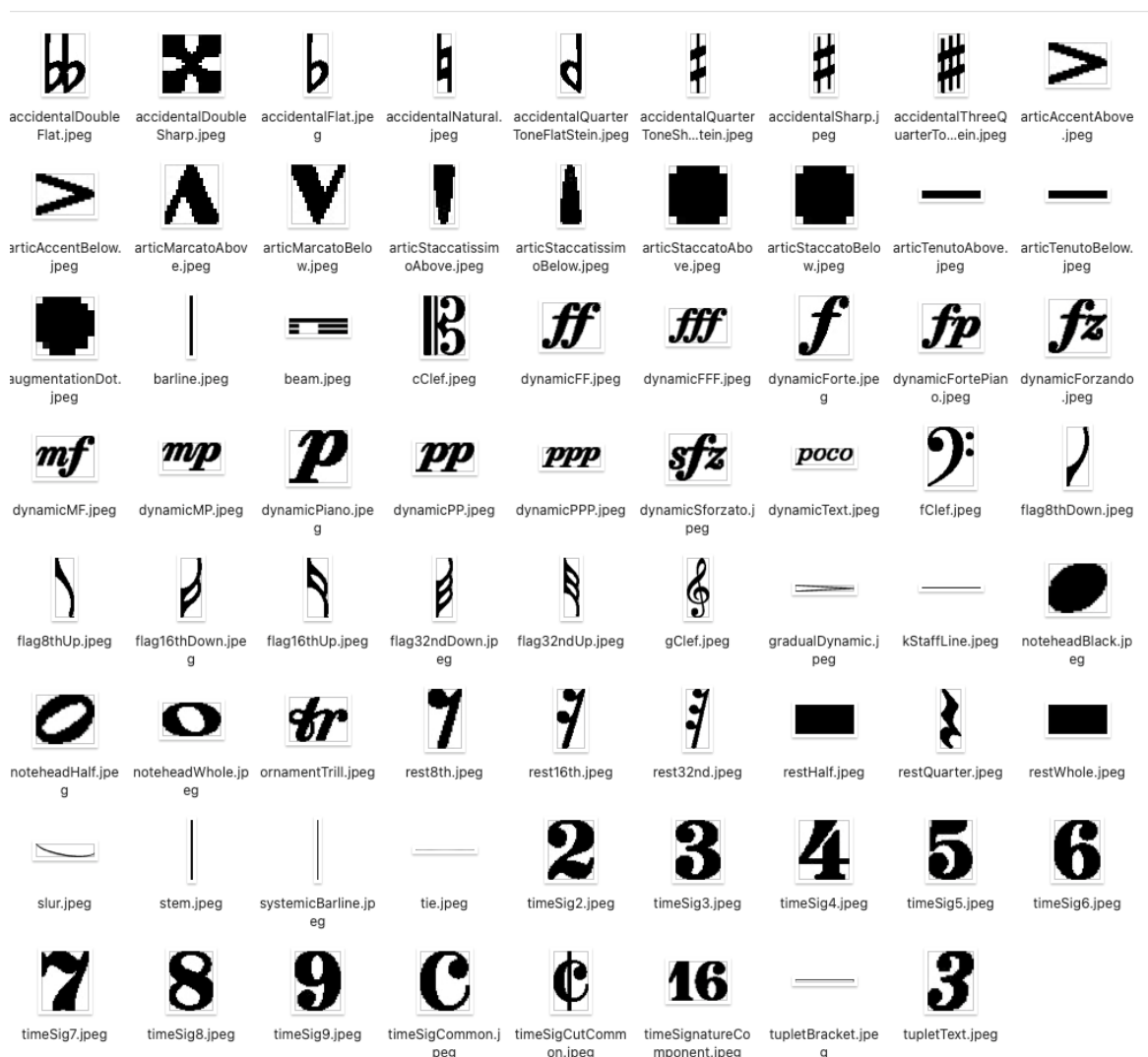


Рисунок 7. Изображения всех 94 классов датасета DOREMi

Для этой работы был использованы данные OMR metadata, программно преобразованные в формат YOLO. Часть классов были объединены, формируя новые классы со следующим распределением:

1. **notehead_filled** (135583) представляет собой noteheadBlack;

2. **notehead_open** (2631) объединяет `noteheadHalf` и `noteheadWhole`;
3. **sharp** (6052) – `accidentalSharp`;
4. **flat** (6056) – `accidentalFlat`;
5. **natural** (4592) – `accidentalNatural`;
6. **double_sharp** (308) – `accidentalDoubleSharp`;
7. **double_flat** (240) – `accidentalDoubleFlat`;
8. **clef** (6173) – `gClef`;
9. **dot** (5453) объединяет объекты-точки: `augmentationDot`, `articStaccatoAbove`, `articStaccatoBelow`, `articStaccatissimoAbove`, `articStaccatissimoBelow`;
10. **stem** (137665) – вертикальные линии: `barline`, `systemicBarline` и `stem`;
11. **beam** (28788) – `beam`;
12. **rest1** (569) – прямоугольные знаки паузы: `restHalf` и `restWhole`;
13. **rest2** (26439) – другие знаки паузы: `rest8th`, `rest16th`, `rest32th`;
14. **other_clef** (1083) `fClef`, `cClef` и `unpitchedPercussionClef1`;
15. **curve** (8667) объединяет `tie`, `slur`, `gradualDynamic` и `tupletBracket`;
16. **other** (15778) – все оставшиеся классы, индивидуально представленные в недостаточном количестве для формирования обучающей выборки.

Исключением является класс **kStaffLine** оригинального набора данных, который не включается в обучение нейросети. Это класс нотных линий, и в разработанной системе для их определения используется детерминированный алгоритм.

В рамках подготовки данных разрешение изображений было понижено, и они были разделены на части 640x640 пикселей (лист. 3). Для каждой части была создана соответствующая разметка, объект включается в разметку, если больше 30% его площади входят в новое изображение.

Листинг 3. Фрагмент кода для разделения каждого изображения на части и создания соответствующих файлов разметки

```
xs = tile_offsets(SCALED_W, TILE_SIZE, TILE_STRIDE)
ys = tile_offsets(SCALED_H, TILE_SIZE, TILE_STRIDE)
```

```

tile_count = 0
for ty in ys:
    for tx in xs:
        tile_name = f"{new_stem}_x{tx:04d}_y{ty:04d}"
        if img is not None:
            tile_img = img[
                ty:ty + TILE_SIZE,
                tx:tx + TILE_SIZE
            ]
            gray = cv2.cvtColor(tile_img,
                                cv2.COLOR_BGR2GRAY)
            black_pixels = np.sum(gray < 128)
            if black_pixels < MIN_BLACK_PIXELS:
                continue
            cv2.imwrite(
                str(img_out_dir / (tile_name + ".jpg")),
                tile_img,
                [cv2.IMWRITE_JPEG_QUALITY, 95]
            )

        yolo_lines = objects_for_tile(objects, tx, ty)
        lbl_file = lbl_out_dir / (tile_name + ".txt")
        with open(lbl_file, "w", encoding="utf-8") as f:
            if yolo_lines:
                f.write("\n".join(yolo_lines) + "\n")

```

Преобразованный датасет содержит 23409 изображений размером 640x640 пикселей, разделенных на тренировочную, валидационную и тестовую выборки в пропорции 8:1:1, соответствующие им файлы разметки, а также данные о нотных линиях в JSON-файлах, сохраненные для проверки работы алгоритма их нахождения. Также сформирован YAML-файл конфигурации для датасета.

5. Обучение модели нейронной сети

Была выбрана нано-версия модели **YOLO26** – самой новой версии YOLO, дающей достаточно высокую точность с ускоренным обучением [6] (табл. 2).

Таблица 2. Полученные значения метрик для модели YOLO26n, обученной на COCO.

Размер	mAP50-95	Скорость (мс) CPU ONNX	Скорость (мс) T4 TensorRT10	Параметры (М)	FLOPs (Б)
640	40,9%	38,9 ± 0,7	1,7 ± 0,0	2,4	5,4

Для подбора оптимальных параметров обучения важно учесть специфику данных, в рассматриваемом случае:

- большое количество мелких объектов на изображении;
- достаточно большой датасет (больше 23 тысяч изображений);
- малое количество шума в данных;
- необходимость сохранять оригинальную ориентацию изображения.

С учетом знаний об используемых данных были сформированы 3 тестовых конфигурации для сравнения метрик после 7 эпох обучения (лист. 4).

Листинг 4. Варианты конфигураций параметров обучения модели для сравнения.

```
CONFIGS = [  
    {"name": "1_pretrained", },  
    {  
        "name": "2_scale03",  
        "scale": 0.3,  
        "box": 10.0,  
        "cos_lr": False,  
    },  
    {  
        "name": "3_scale02",  
        "scale": 0.2,  
        "box": 10.0,  
        "cos_lr": True,  
        "hsv_v": 0.5,  
    },  
]
```

```

{
    "name": "4_scale01",
    "scale": 0.1,
    "box": 12.0,
    "cos_lr": True,
    "hsv_v": 0.5,
    "weight_decay": 3e-4,
},
]

```

Для определения базового значения были проведены тесты на модели дообучения на датасете и с тремя разными конфигурациями. В листинге 4 перечислены аргументы, отличающие конфигурации. Были выбраны следующие параметры:

- **scale** – влияет на случайное масштабирование объектов при обучении, понижено для уменьшения ошибки с объектами разного размера, но похожей формы;
- **mosaic** – вероятность объединения изображений при обучении;
- **box** – параметр, регулирующий важность точного определения границ bounding box, влияет на локализацию объектов;
- **cos_lr** – регулирует использование косинусного планировщика скорости обучения, уменьшающего её плавнее, чем при линейном затухании;
- **hsv_v** – модифицирует яркость изображения, помогая модели работать с изображениями разного освещения;
- **weight_decay** – коэффициент L2 регуляризации, штрафующий большие веса для предотвращения переобучения, может быть снижен в больших датасетах [7].

Часть параметров были общими для всех вариантов, они приведены в листинге 5:

- **data** – путь к YAML-файлу датасета;
- **imgsz** – размер изображения, подаваемого на вход нейросети;
- **batch** – размер пакета данных, обрабатываемого за одну итерацию обучения;
- **epochs** – количество эпох обучения;

- **warmup_epochs** – количество эпох обучения со сниженной скоростью обучения;
- **device** – определяет аппаратное устройство, используемое при обучении, 0 – CPU;
- **seed** – начальное значение генератора псевдослучайных чисел для воспроизводимости обучения с теми же параметрами;
- **verbose** – флаг, определяющий детальность вывода отладочной информации в процессе обучения;
- **cls** – влияет на то, насколько сильно модель штрафует за ошибку в определении класса объекта;
- **fliplr** – вероятность применения горизонтального отражения изображения [7].

Листинг 5. Общие параметры для всех тестовых конфигураций.

```
COMMON = dict(
    data = DATA,
    imgsz = 640,
    batch = 16,
    epochs = 7,
    warmup_epochs = 2,
    device = 0,
    seed = 42,
    verbose = False,
    cls = 1.0,
    fliplr = 0.0,
)
```

Конфигурация 3 показала лучшие результаты и была использована для обучения (табл. 3).

Таблица 3. Значения метрик для разных конфигураций.

Метрики	Конфигурация 1	Конфигурация 2	Конфигурация 3
mAP50	69,82%	70,09%	70,80%
mAP50-95	54,80%	54,49%	55,45%
Precision	61,90%	61,94%	64,61%
Recall	76,02%	77,01%	76,20%
F1	68,23%	68,66%	69,93%

Обучение происходило с использованием видеокарты NVIDIA GeForce GTX 1660 Ti 6GB, код загрузки и обучения модели приведен в листинге 6.

Листинг 6. Загрузка и обучение модели

```
model = YOLO("yolo26n.pt")
results = model.train(
    data="data.yaml",
    imgsz = 640,
    batch = 16,
    device = 0,
    exist_ok=True,
    name="final",
    epochs = 80,
    patience=15,
    warmup_epochs = 4,
    seed = 42,
    verbose = False,
    scale = 0.1,
    box = 12.0,
    cos_lr = True,
    hsv_v = 0.5,
    weight_decay = 3e-4,
    fliplr=0.0,
    cls=1.0,
)
```

Результаты обучения модели

Обучение было остановлено с помощью механизма ранней остановки на 37 эпохе, с лучшим результатом на эпохе 22. Семейство моделей YOLO предоставляет автоматически сформированные графики метрик, они представлены на рисунке 8.

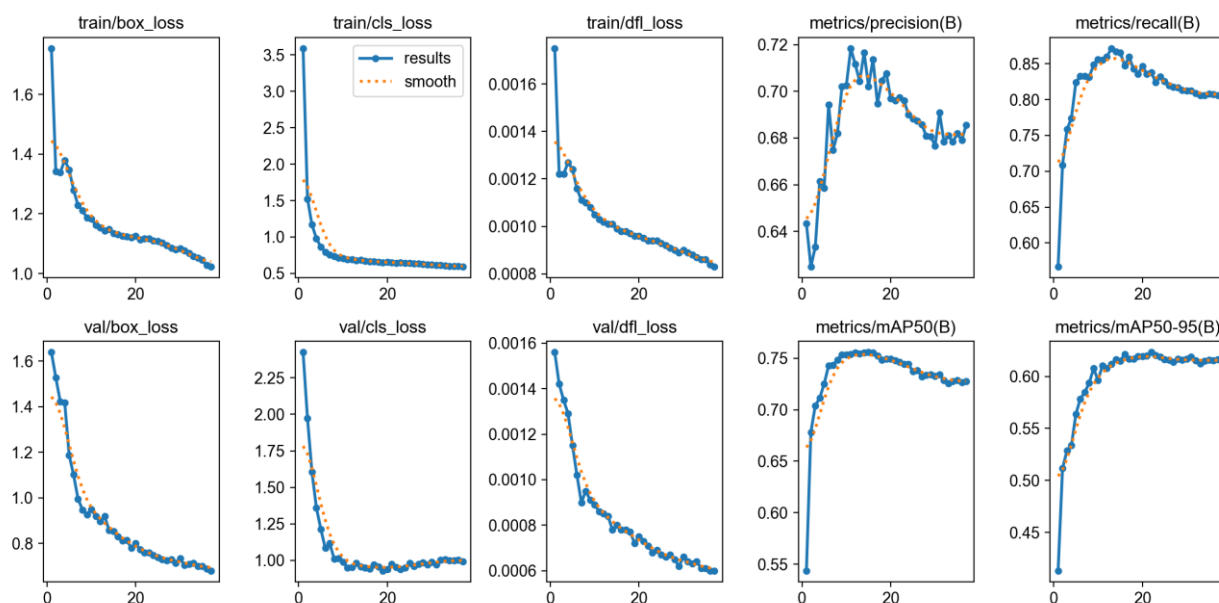


Рисунок 8. Результаты обучения модели YOLO26

Эти графики включают следующие метрики:

- **box_loss** – потеря, связанная с предсказанием ограничивающих рамок;
- **cls_loss** – потеря, связанная с предсказанием классов объектов;
- **dfl_loss** – потеря, связанная с функцией потерь, типично фокальной функцией потерь (Distribution Focal Loss);

Все графики потерь демонстрируют снижение как на тренировочной, так и на валидационной выборках, без переобучения.

- **precision** – показывает, какая доля предсказанных моделью положительных результатов предсказана верно;
- **recall** – показывает, какая доля объектов были корректно предсказаны моделью;
- **mAP50** – Mean Average Precision – стандартная метрика при детекции объектов, которая объединяет precision, recall с IoU (Intersection over Union, мера качества локализации) – при пороге IoU=0.5;
- **mAP50-95** – Mean Average Precision при порогах IoU от 0.5 до 0.95 с шагом 0.05.

Графики метрик качества показывают улучшение в процессе обучения и стабилизируются на определенном уровне.

На тестовой выборке удалось достигнуть точности 63,2% (mAP50-95), полученные значения различных метрик представлены в таблице 4.

Таблица 4. Значения метрик на тестовом наборе (3288 изображений, 138254 объекта)

Precision	Recall	mAP50	mAP50-95
69,8%	83,7%	74,6%	63,2%

6. Интеграция результатов детекции нот и линий для формирования списка нот

На основе обученной нейросети и с использованием разработанного алгоритма была сформирована система, извлекающая из изображения, поданного на вход информацию о нотах на нем. Обработка состоит из следующих шагов:

1. Изображение подается на вход, из него извлекается информация о линиях стана, также в процессе происходит коррекция возможного наклона, и на основе этих данных размер изображения масштабируется так, чтобы расстояние между линиями нотного стана соответствовало таковому в наборе данных для обучения нейросети. Это делается для стабилизации размеров знаков музыкальной нотации.
2. Изображение разрезается на части 640x640 пикселей со сдвигом 400 и с сохранением сдвига каждой части относительно оригинала. Каждая из этих частей обрабатывается нейронной сетью для получения данных об объектах на ней. Далее предсказания пересчитываются обратно в глобальные координаты для сопоставления с нотными линиями и обрабатываются алгоритмом подавления немаксимума.

Также на этом этапе происходит отбор рабочих классов, оставляя классы 0-7 (ноты, знаки альтерации и скрипичный ключ).

Подавление немаксимума (Non-Maximum Suppression, NMS) – алгоритм постобработки предсказаний нейросети, предназначенный для удаления избыточной дублирующейся разметки одного объекта. Выбирает ограничивающие рамки с наибольшей уверенностью и удаляет те, у которых пересечение с ними превышает определенный порог.

3. Обнаруженные ранее линии разделяются по группам с учетом расстояния между ними. Образованные группы длиной 5 считаются нотными станами.
4. Все объекты распределяются по станам, привязываясь к ближайшему по высоте и внутри этого распределения сортируются по горизонтальному порядку. Они дополнительно разделяются по классам на 3 группы: ноты, знаки альтерации и скрипичные ключи. Происходит отдельная детекция знаков при ключе (знаков, действующих на весь стан) и после этого рассчитывается высота каждой ноты (каждой ноте присваивается название, содержащее ноту и октаву), при этом учитываются как знаки при ключе, так и случайные знаки альтерации. Для снижения влияния ошибочных детекций символов на листе, те объекты, которые находятся за пределами возможных местоположений нот, игнорируются.

Определение высоты нот

Основная работа этого этапа выполняется в функции **compute_pitch_with_acc**. Эта функция принимает на вход объект ноты, объект нотного стана, содержащий информацию о привязанных к нему символах, а также список знаков при ключе. Вычисляется позиция объекта относительно линий нотного стана, производится поиск случайных знаков альтерации, и далее ноте назначается знак, который на неё влияет. Приоритет имеет локальный, далее ключевой знаки, при их отсутствии указывается, что нота имеет оригинальную высоту (лист. 7).

Листинг 7. Функция вычисления высоты ноты

```
def compute_pitch_with_acc(  
    note: Detection,  
    staff: Staff,  
    key_sig: dict,  
) -> tuple:  
    pos = symb_position(note, staff)  
    local = find_local_acc(note, staff, pos)  
  
    if local is not None:  
        applied = local.cls
```

```

elif pos in key_sig:
    applied = key_sig[pos]
else:
    applied = None
base_name = PITCH_MAP.get(pos, f"pos={pos}")
if applied is not None:
    symbol = ACCIDENTAL_SYMBOL.get(applied, '')
    pitch = base_name + symbol
else:
    pitch = base_name
is_valid = pos in PITCH_MAP
return pitch, pos, applied, is_valid

```

Данные о соответствии позиций нот на стане названиям нот получаются с помощью словаря PITCH_MAP, где ключ – позиция на стане, а значение – название ноты (например, 0: "E4"). Позиции нот в нем ограничены нижней и верхней нотами, доступными в классическом строе шестиструнной гитары.

7. Алгоритм построения аппликатуры для шестиструнной гитары методом динамического программирования

Некоторые музыкальные термины, которые будут использованы в этой главе:

Аккорд – одновременное звучание трёх и более нот (в рамках этой работы двух и более нот), в гитарной технике реализуется прижатием нескольких струн на заданных ладах и совместным звукоизвлечением.

Аппликатура – это указание, каким образом играть на инструменте ту или иную ноту, преимущественно используется со струнными инструментами.

Лад – элемент конструкции грифа, металлическая перегородка, определяющая высоту звучания струны при прижатии; номер лада, на котором зажимается струна.

Позицией называют расположение руки гитариста на грифе, обычно по верхнему ладу, над которым находится указательный палец. Определяет совокупность нот, доступных без смещения кисти.

Табулатура – система графической нотации для струнных щипковых инструментов, в которой горизонтальные линии соответствуют струнам, а числа на них – номерам прижимаемых ладов.

Особенность струнных инструментов, в частности рассматриваемой гитары, в том, что большинство нот можно извлечь несколькими способами. Это сильно усложняет автоматизацию создания аппликатур, так как программа должна понимать, какой вариант стоит выбрать в каждом отдельном случае.

Когда гитарист первый раз разбирает новое произведение, он принимает решение на каких ладах и струнах будут исполняться отдельные ноты. При этом необходимо учитывать физическую возможность сыграть записанный аккорд и избегать большого количества «прыжков» по грифу. Однако, иногда для обеспечения плавного перехода к высокой ноте, исполнение которой на

данной позиции затруднительно или невозможно, необходимо заранее переместить руку в иную позицию.

При разработке подсистемы, способной выполнить вышеописанную задачу, было принято решение воспользоваться методом динамического программирования – алгоритмом Витерби.

Алгоритм Витерби – это алгоритм динамического программирования для поиска оптимальной последовательности состояний в задаче, представимой в виде ориентированного графа без циклов, где вершины соответствуют состояниям на каждом временном шаге, а рёбра имеют заданные веса. В данной работе состояниями выступают различные положения руки на грифе, а как временной шаг рассматриваются группы нот.

В качестве функции стоимости для каждого шага используется сумма стоимости текущей позиции и перемещения в новую позицию.

Стоимость текущей позиции рассчитывается как сумма прижатых ладов, умноженная на небольшой коэффициент, это мотивирует алгоритм незначительно приоритизировать более высокие позиции на грифе.

Функция оценки стоимости перемещения между позициями основана на следующих принципах:

- переход в позицию или из позиции с открытыми струнами не влияет на расположение руки – нулевой штраф;
- перемещение в пределах 3 ладов дает небольшой штраф;
- при превышении порога в 3 лада стоимость прыжка резко возрастает с увеличением расстояния, что поощряет ранний и более плавный переход на соседние струны (лист. 8).

Листинг 8. Функция стоимости перемещения для алгоритма Витерби.

```
def move_cost(fret_a: int, fret_b: int) -> float:
    if fret_a == 0 or fret_b == 0:
        return 0.0
    dist = abs(fret_b - fret_a)
    if dist == 0:
        return 0.0
    if dist <= 3:
        return dist * 0.4
```

```
return 3 * 0.4 + (dist - 3) ** 2 * 0.6
```

Алгоритм использует карту грифа гитары, в которой для каждой из шести струн задано соответствие между номером лада и полным названием ноты. На основе этой карты строится словарь `_MIDI_TO_POSITIONS`, в котором ключами являются численные значения высоты нот по аналогии с форматом MIDI, а значениями – все возможные сочетания позиций струна-лад, соответствующие этой ноте.

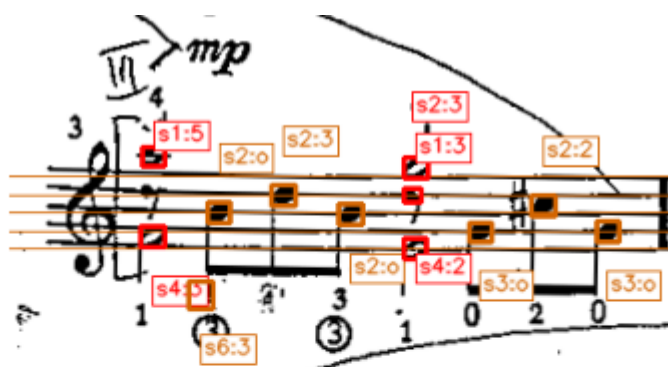
Обработка ведется по станам, последовательность нот группируется в события – одиночные ноты или аккорды. Аккордами считаются ноты, у которых разница горизонтальных координат не превышает порога. Внутри группы ноты сортируются по вертикали.

Для каждого события перебором находятся все возможные комбинации струна-лад. Для одиночной ноты возвращаются значения из `_MIDI_TO_POSITIONS`, но для аккорда – группы нот – накладываются дополнительные ограничения:

- каждая струна используется не более одного раза;
- разница между минимальным и максимальным прижатыми ладами не должна превышать заданный порог (открытые струны не учитываются) – это нужно для исключения физически невозможных аккордов.

После этого с помощью алгоритма Витерби строится оптимальный путь, на основе которого формируется список объектов **GuitarPlacement**, готовых к визуализации.

Для демонстрации была построена табулатура, представленная на рисунке 9. Буквы E, A, D, G, B, e обозначают струны от шестой до первой и для каждой обнаруженной ноты или аккорда указано, на каких ладах их стоит играть.



Табулатура:

	A5+F4	G3	B4	D5	B4	G5+D5+E4	G4	C5#	G4
e	5	----	----	----	----	3	----	----	----
B	----	----	0	3	0	3	----	2	----
G	----	----	----	----	----	----	0	----	0
D	3	----	----	----	----	2	----	----	----
A	----	----	----	----	----	----	----	----	----
E	----	3	----	----	----	----	----	----	----

Рисунок 9. Фрагмент нотного листа с рекомендациями системы по струне и ладу и соответствующая табулатура.

Использование ИИ в работе

В процессе выполнения выпускной квалификационной работы в качестве вспомогательного инструмента применялись генеративные языковые модели – **Claude 3.6 Sonnet** и **DeepSeek**.

Конкретные области применения:

1. **Помощь с формулировками** – использование ИИ для редакции текста, а именно для уточнения академических формулировок и устранения стилистических неточностей.
2. **Генерация кода демонстрационных функций** – помощь при реализации вспомогательных фрагментов программного кода для демонстрации результатов: отрисовка отметок, обозначающих полученные ноты, а также рекомендации системы на изображении, печать таблиц с результатами. В частности, в функциях `print_notes`, `visualize_notes`, `visualize_guitar_fingering`.

Архитектура разработанной системы, алгоритмы и ключевые решения спроектированы и реализованы без привлечения средств генеративного ИИ.

Заключение

В ходе выполнения данной работы была разработана интеллектуальная система на основе нейронных сетей, способная распознавать нотные партитуры для гитары по изображениям и формировать аппликатуру для них.

Были выполнены все поставленные задачи:

1. Проанализированы существующие методы для распознавания нотных партитур по изображениям.
2. Разработана архитектура системы, сочетающей детерминированные алгоритмы с нейронными сетями.
3. Реализован алгоритм для определения позиций нотных линий на изображении.
4. Выбран и предобработан датасет, содержащий музыкальную нотацию.
5. Обучена модель нейронной сети для распознавания музыкальных символов на изображении.
6. Разработан алгоритм для интеграции данных, извлеченных из изображения, и формирования списка нот.
7. Разработана подсистема, формирующая аппликатуру из полученного списка нот.

Код тестирования алгоритмов, модели, а также код и другие составляющие разработанной системы расположены по ссылке:

https://github.com/apushist/VKR_Pushkova

Список литературы:

1. Alexander Pacha. Self-learning optical music recognition – Vienna, 18.06.2019
2. Capella Software. Eighth rest or smudge. (дата обращения: 27.04.2026)
URL: <https://www.capella-software.com/us/index.cfm/products/capella-scan/eighth-rest-or-smudge/>
3. Документация Audiveris. Описание проекта. (дата обращения: 05.05.2026)
URL: <https://github.com/Audiveris/audiveris/blob/master/README.md>
4. Документация Audiveris. Driving the pipeline.
URL:
<https://audiveris.github.io/audiveris/pages/guides/main/pipeline/#driving-the-pipeline>
5. Документация датасета DOREMi. (дата обращения: 20.04.2026)
URL: <https://github.com/steinbergmedia/DoReMi/>
6. Документация Ultralytics. Ultralytics YOLO26. (дата обращения: 05.05.2026)
URL: <https://docs.ultralytics.com/ru/models/yolo26>
7. Документация Ultralytics. Конфигурация. (дата обращения: 27.04.2026)
URL:
https://docs.ultralytics.com/ru/usage/cfg?h=show_labels#augmentation-settings